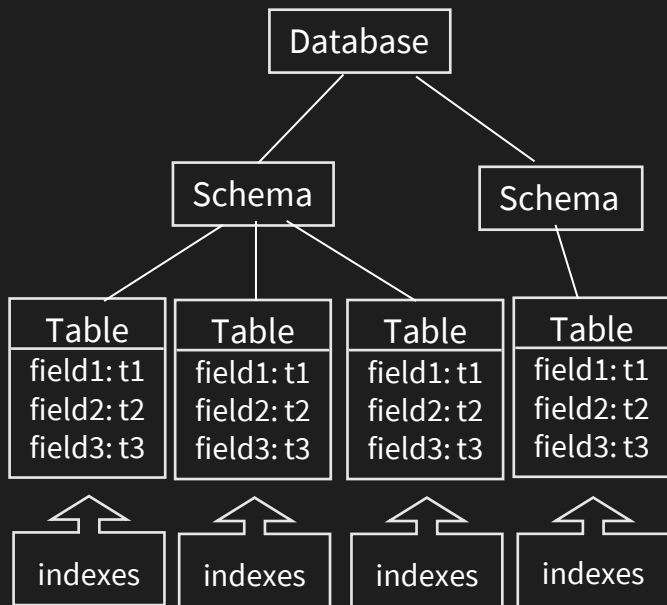




OVERVIEW

SHENKAR



מבנה בסיס נתונים

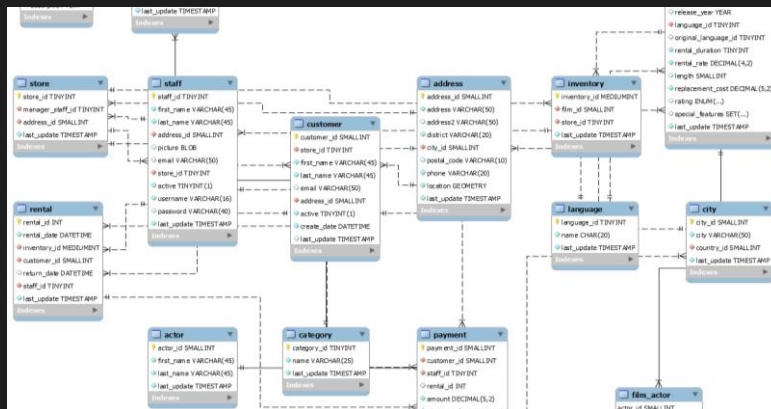
בסיס נתונים מכיל סכימה,
המתארת את מבנהו.

סכימה יכולה להכיל:

tables - containing data

index files - for fast lookup of
data

stored procedures, constraints,
triggers, and more



מבנה של טבלה

Every field has:

- a name
- a data type and length

To view the structure of a table use:

```
DESCRIBE tablename
```

מבנה טבלה

Every column has:

- a name
- a data type and length

To view the structure of a table use:

`DESCRIBE tablename`

describe city;						
Result Grid						
Filter Rows:		Export:		Wrap Cell Contents:		
Field	Type	Null	Key	Default	Extra	
ID	int	NO	PRI		auto_increment	
Name	char(35)	NO				
CountryCode	char(3)	NO	MUL			
District	char(20)	NO				
Population	int	NO		0		

שפת ה - SQL

- DDL - Data Definition Language - משמשת להגדרות ויצירת סכמה
 - Create database
 - Create, Drop, Alter tables
- DML - Data Manipulation Language - משמשת לעבודה עם הנתונים שבטבלה
 - Pull data.
 - Insert new data
 - Update
 - Delete
- DCL - Data Control Language - משמשת לפעולות תחזוקה, הגדרות והרשאות

שליפת נתונים

Select columns from a table that match some criteria:

```
SELECT field1, field2, field3  
FROM table  
WHERE condition  
ORDER BY field1,... [ASC|DESC];
```

Example: cities with population > 5 M

```
SELECT * FROM City  
WHERE population > 5000000  
ORDER BY population DESC;
```

דוגמאות לשליפה

- `select * from actor; -- select all columns from table`
- `select first_name, last_name from actor; -- select specific columns by comma`
- `select first_name fname from actor; -- alias`
- `select distinct first_name from actor; -- remove duplicate`
- `select distinct first_name from actor order by first_name; -- order resultset by name ;`

דוגמאות לשליפה

- `select * from customer where store_id = 1;`
- `select * from customer where store_id = 1 and first_name = 'LINDA';`
- `select * from customer where store_id = 1 and first_name like 'mar%';`
- `select address_id,first_name,last_name from customer where address_id = 5 or address_id = 11;`
- `select address_id,first_name,last_name from customer where address_id in (5,11);`
- `select title from film where length = (select min(length) from film);`

שליפה עם שאילתה מקוננת

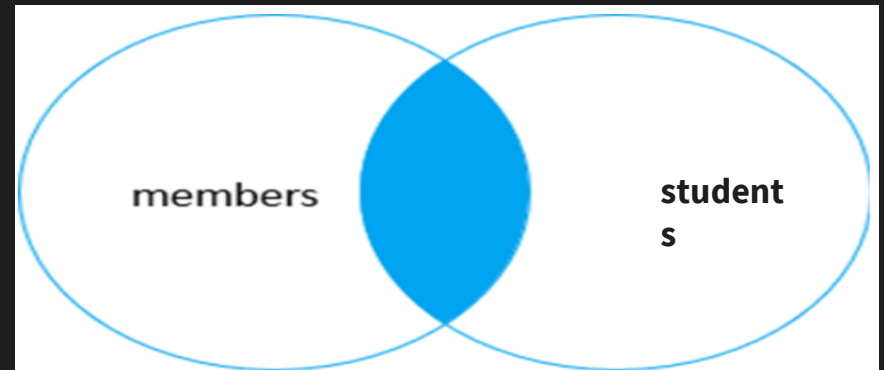
- ```
SELECT * FROM (SELECT actor.actor_id,
actor.first_name, actor.last_name, count(*)
num_of_films FROM actor INNER JOIN
film_actor fa ON actor.actor_id = fa.actor_id INNER
JOIN film ON film.film_id = fa.film_id GROUP BY
actor.actor_id, actor.first_name, actor.last_name
) as f WHERE num_of_films > 5; ;
```

# חיבור טבלאות

## INNER JOIN

The following statement finds members who are also students:

```
SELECT
 m.member_id,
 m.name memmber_name,
 c.student_id,
 c.name student_name
FROM members AS m
INNER JOIN students AS c
ON m.name = c.name;
```



|   | member_id | memmber_name | student_id | student_name |
|---|-----------|--------------|------------|--------------|
| ▶ | 1         | John         | 1          | John         |
|   | 3         | Mary         | 2          | Mary         |
|   | 5         | Amelia       | 3          | Amelia       |

# חיבור טבלאות

## LEFT JOIN

SELECT

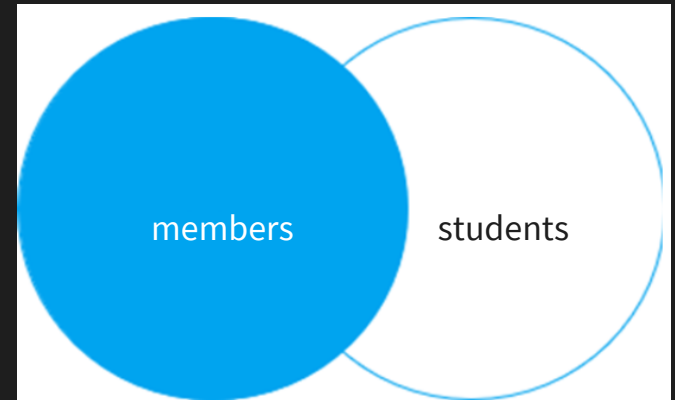
m.member\_id,  
m.name AS member,  
c.student\_id,  
c.name AS 'student name'

FROM

members m

left join

students c using(name);

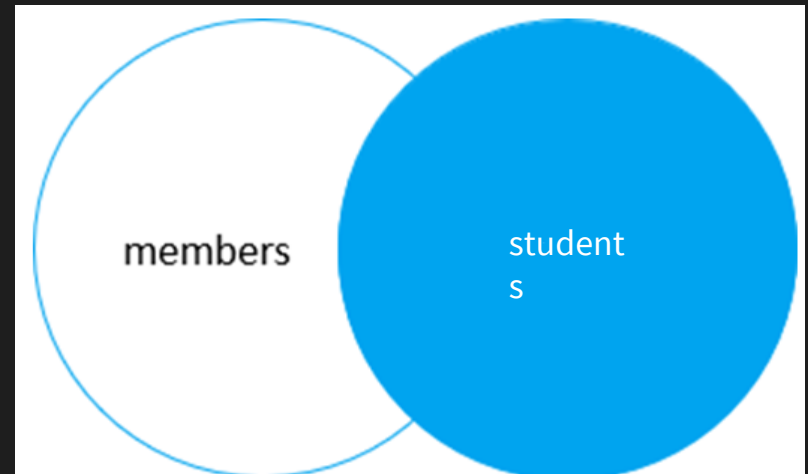


|   | member_id | member | student_id | student name |
|---|-----------|--------|------------|--------------|
| ▶ | 1         | John   | 1          | John         |
|   | 3         | Mary   | 2          | Mary         |
|   | 5         | Amelia | 3          | Amelia       |
|   | 2         | Jane   | NULL       | NULL         |
|   | 4         | David  | NULL       | NULL         |

# חיבור טבלאות

## RIGHT JOIN

```
SELECT
 m.member_id,
 m.name as member,
 c.student_id, c.name
 student
FROM
 members m
right join students c
on c.name = m.name;
```



|   | member_id | member | student_id | student |
|---|-----------|--------|------------|---------|
| ▶ | 1         | John   | 1          | John    |
|   | 3         | Mary   | 2          | Mary    |
|   | 5         | Amelia | 3          | Amelia  |
|   | NULL      | NULL   | 4          | Joe     |

## פונקציות חישוביות וקיבוץ נתונים

Functions can have *arguments*, just like C, Java, etc.

**SUM**(*expression*)

*SELECT SUM*(length) *FROM film*;

**AVG**(*expression*)

*SELECT AVG*(length) *FROM film*;

**MAX**(*expression*)

*SELECT MAX*(length) *FROM film*;

**MIN**(*expression*)

*SELECT MIN*(length) *FROM film*;

**COUNT**(*expression*)

*SELECT COUNT*(\*) *FROM film*;

## GROUP BY ... קיבוץ

- **GROUP BY** . . . is used when you want to apply a function (count, sum, avg) to **a group of rows** having a common characteristic.
- example: How many cities are in each country?

```
SELECT country_id,COUNT(*)
number_cities_in_country FROM city GROUP
BY country_id;
```

add HAVING to restrict results to count( ) > 10

```
SELECT special_features,COUNT(*) FROM
sakila.film GROUP BY special_features
HAVING COUNT(*) > 70;
```

## יצירת טבלה

- create table without data:
- *CREATE TABLE table (column\_name column\_type [(length)] [NOT NULL],...);*
- *CREATE TABLE EXAMPLE*  
*(*
  - *AB VARCHAR(15),*  
*CD INTEGER NOT NULL,*
  - *EF DECIMAL(5),*  
*GH DECIMAL(7,2)*
  - ***PRIMARY KEY (AB,CD)****);*

create table from query (with or without data):

*CREATE TABLE table AS SELECT ...;*

```
create table actor_as_select
select * from sakila.actor limit 10;
```

## שינוי במבנה טבלה

- add column:

- *alter table actor\_as\_select\_3 add b\_day date;*

- drop column:

- *alter table actor\_as\_select\_3 drop b\_day;*

- change column type:

*alter table actor\_as\_select\_3 modify lname varchar(55);*



## מחיקת טבלה

- remove table:
- drop table table\_name;
- *drop table actor\_as\_select\_3;*

## הכנסת מידע לטבלה

- INSERT INTO tablename [( column-list )] VALUES ( constant-list ) ;

| actor_id | first_name | last_name    | last_update         |
|----------|------------|--------------|---------------------|
| 1        | PENELOPE   | GUINESS      | 2006-02-15 04:34:33 |
| 2        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
| 3        | ED         | CHASE        | 2006-02-15 04:34:33 |
| 4        | JENNIFER   | DAVIS        | 2006-02-15 04:34:33 |
| 5        | JOHNNY     | LOLLOBRIGIDA | 2006-02-15 04:34:33 |
| 6        | BETTE      | NICHOLSON    | 2006-02-15 04:34:33 |
| 7        | GRACE      | MOSTEL       | 2006-02-15 04:34:33 |
| 8        | MATTHEW    | JOHANSSON    | 2006-02-15 04:34:33 |
| 9        | JOE        | SWANK        | 2006-02-15 04:34:33 |
| 10       | CHRISTIAN  | GABLE        | 2006-02-15 04:34:33 |

| actor_id | first_name | last_name    | last_update         |
|----------|------------|--------------|---------------------|
| 1        | PENELOPE   | GUINESS      | 2006-02-15 04:34:33 |
| 2        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
| 3        | ED         | CHASE        | 2006-02-15 04:34:33 |
| 4        | JENNIFER   | DAVIS        | 2006-02-15 04:34:33 |
| 5        | JOHNNY     | LOLLOBRIGIDA | 2006-02-15 04:34:33 |
| 6        | BETTE      | NICHOLSON    | 2006-02-15 04:34:33 |
| 7        | GRACE      | MOSTEL       | 2006-02-15 04:34:33 |
| 8        | MATTHEW    | JOHANSSON    | 2006-02-15 04:34:33 |
| 9        | JOE        | SWANK        | 2006-02-15 04:34:33 |
| 10       | CHRISTIAN  | GABLE        | 2006-02-15 04:34:33 |
| 11       | Gal        | Omer         | 2006-02-14 00:00:00 |

*insert into actor\_as\_select(actor\_id,first\_name, last\_name, last\_update)  
values(11,'Gal','Omer','2006-02-14');*

- We can also insert data to table based on select.

```
insert into actor_as_select
select * from actor_as_select limit 3;
```

|   | actor_id | first_name | last_name    | last_update         |
|---|----------|------------|--------------|---------------------|
|   | 1        | PENELOPE   | GUINESS      | 2006-02-15 04:34:33 |
|   | 2        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
|   | 3        | ED         | CHASE        | 2006-02-15 04:34:33 |
|   | 4        | JENNIFER   | DAVIS        | 2006-02-15 04:34:33 |
|   | 5        | JOHNNY     | LOLLOBRIGIDA | 2006-02-15 04:34:33 |
|   | 6        | BETTE      | NICHOLSON    | 2006-02-15 04:34:33 |
|   | 7        | GRACE      | MOSTEL       | 2006-02-15 04:34:33 |
|   | 8        | MATTHEW    | JOHANSSON    | 2006-02-15 04:34:33 |
| ▶ | 9        | JOE        | SWANK        | 2006-02-15 04:34:33 |
|   | 10       | CHRISTIAN  | GABLE        | 2006-02-15 04:34:33 |
|   | 11       | Gal        | Omer         | 2006-02-14 00:00:00 |
|   | 12       | Stas       | K            | 2021-04-01 20:51:46 |
|   | 1        | PENELOPE   | GUINESS      | 2006-02-15 04:34:33 |
|   | 2        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
|   | 3        | ED         | CHASE        | 2006-02-15 04:34:33 |

## עדכון נתונים

- UPDATE tablename SET column-assignment-list  
WHERE conditional-expression ;
- *update actor\_as\_select  
set first\_name = 'EDI', last\_update = now()  
where actor\_id = 3;*

|  | actor_id | first_name | last_name | last_update         |
|--|----------|------------|-----------|---------------------|
|  | 1        | PENELOPE   | GUINESS   | 2006-02-15 04:34:33 |
|  | 2        | NICK       | WAHLBERG  | 2006-02-15 04:34:33 |
|  | 3        | ED         | CHASE     | 2006-02-15 04:34:33 |
|  | 4        | JENNIFER   | DAVIS     | 2006-02-15 04:34:33 |



|   | actor_id | first_name | last_name | last_update         |
|---|----------|------------|-----------|---------------------|
| ▶ | 1        | PENELOPE   | GUINESS   | 2006-02-15 04:34:33 |
|   | 2        | NICK       | WAHLBERG  | 2006-02-15 04:34:33 |
|   | 3        | EDI        | CHASE     | 2021-04-01 21:23:07 |
|   | 4        | JENNIFER   | DAVIS     | 2006-02-15 04:34:33 |

## מחיקת נתונים

- DELETE FROM tablename  
WHERE conditional-expression ;

*delete from actor\_as\_select  
where actor\_id = 2;*

| actor_id | first_name | last_name    | last_update         |
|----------|------------|--------------|---------------------|
| 2        | PENELOPE   | GUINESS      | 2006-02-15 04:34:33 |
| 2        | PENELOPE   | GUINESS      | 2006-02-15 04:34:33 |
| 4        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
| 4        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
| 6        | EDI        | CHASE        | 2006-02-15 04:34:33 |
| 6        | EDI        | CHASE        | 2006-02-15 04:34:33 |
| 8        | JENNIFER   | DAVIS        | 2006-02-15 04:34:33 |
| 10       | JOHNNY     | LOLLOBRIGIDA | 2006-02-15 04:34:33 |
| 12       | BETTE      | NICHOLSON    | 2006-02-15 04:34:33 |
| 14       | GRACE      | MOSTEL       | 2006-02-15 04:34:33 |
| 16       | MATTHEW    | JOHANSSON    | 2006-02-15 04:34:33 |
| 18       | JOE        | SWANK        | 2006-02-15 04:34:33 |
| 20       | CHRISTIAN  | GABLE        | 2006-02-15 04:34:33 |
| 22       | Gal        | Omer         | 2006-02-15 04:34:33 |
| 24       | Stas       | K            | 2006-02-15 04:34:33 |



| actor_id | first_name | last_name    | last_update         |
|----------|------------|--------------|---------------------|
| 4        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
| 4        | NICK       | WAHLBERG     | 2006-02-15 04:34:33 |
| 6        | EDI        | CHASE        | 2006-02-15 04:34:33 |
| 6        | EDI        | CHASE        | 2006-02-15 04:34:33 |
| 8        | JENNIFER   | DAVIS        | 2006-02-15 04:34:33 |
| 10       | JOHNNY     | LOLLOBRIGIDA | 2006-02-15 04:34:33 |
| 12       | BETTE      | NICHOLSON    | 2006-02-15 04:34:33 |
| 14       | GRACE      | MOSTEL       | 2006-02-15 04:34:33 |
| 16       | MATTHEW    | JOHANSSON    | 2006-02-15 04:34:33 |
| 18       | JOE        | SWANK        | 2006-02-15 04:34:33 |
| 20       | CHRISTIAN  | GABLE        | 2006-02-15 04:34:33 |
| 22       | Gal        | Omer         | 2006-02-15 04:34:33 |
| 24       | Stas       | K            | 2006-02-15 04:34:33 |

# Constraints - אילוצים

- **Primary Key**
  - No two rows can have same values for primary key
  - A table can have only one primary key
- **NOT NULL**
- **UNIQUE**
- **CHECK**
- **Foreign Key**
-

## TRIGGERS

- Specify automatic actions that database system will perform when certain events and conditions occur

```
CREATE TRIGGER ins_employee AFTER INSERT ON
employee FOR EACH ROW BEGIN
INSERT INTO health_plan (emp_ssn,provider)
VALUES (new.emp_ssn, 'AI') ;
END;
```

## מבט - SQL VIEW

- A SQL view to show actors and their films:

```
CREATE VIEW actor_film AS
SELECT
actor.first_name, actor.last_name, film.title film_id
FROM
actor
INNER JOIN
 film_actor fa
ON actor.actor_id = fa.actor_id
INNER JOIN
film
 ON film.film_id = fa.film_id;
```

- Views can take arguments and return values



## דוגמאות לשימוש ב VIEW

- Can use our view for simple select:

```
SELECT * from actor_film;
```

- Can include in join queries:

```
SELECT ac.*, c.name category
FROM actor_film ac
INNER JOIN film_category fa ON ac.film_id = fa.film_id
INNER JOIN category c ON fa.category_id = c.category_id ;
```

- Can include in **WHERE** clause:

```
SELECT first_name, last_name,
COUNT(*) num_of_film
FROM actor_film
WHERE
first_name = 'KEVIN'
GROUP BY first_name , last_name;
```

## פונקציות משתמש

- A SQL function to count how many films a particular actor has:

```
CREATE FUNCTION
 actor_count(in_first_name VARCHAR(45),in_last_name
 VARCHAR(45)) RETURNS INTEGER
BEGIN
 DECLARE a_count INTEGER;
 SELECT COUNT(*) INTO a_count
 FROM actor inner join
 film_actor fa
 on
 actor.actor_id = fa.actor_id
 WHERE actor.first_name = in_first_name and actor.last_name =
 in_last_name ;
 RETURN a_count;
END
```

- Function can take arguments and return values
- Use SQL statements and other operations in body

## דוגמאות לשימוש בפונקציות

- Can use our function for individual actor:

```
SELECT actor_count('NICK', 'WAHLBERG');
```

- Can include in computed results:

```
SELECT first_name, last_name ,
actor_count(first_name, last_name) AS number_of_film
FROM actor;
```

- Can include in **WHERE** clause:

```
SELECT first_name, last_name
 ,actor_count(first_name,
last_name) AS number_of_film
FROM actor WHERE
actor_count(first_name,
last_name) BETWEEN 20 AND 25;
```

## פרוצדורות

```
CREATE PROCEDURE category_summary(IN
 in_category_name VARCHAR(20))
BEGIN
 SELECT c.name category_name,
 count(*) num_of_movies,
 sum(film.length) total_length FROM
 film INNER JOIN
 film_category fc ON film.film_id =
 fc.film_id INNER JOIN
 category c ON fc.category_id =
 c.category_id WHERE
 c.name = in_category_name
 GROUP BY C.NAME;
END
```

## דוגמאות לשימוש בפרוצדורות

```
CALL category_summary('action');
CALL category_summary_out('action',@cat,@num,@length);
```

```
select @cat,@num,@length;
```

```
@cat, @num, @length
'Action', '64', '7143'
```

# תכנון ועיצוב בסיס נתונים – THE ER MODEL

- The ER data model employs three basic concepts:
  - entity sets
  - relationship sets
  - Attributes
- The ER model also has an associated diagrammatic representation, the ER diagram (ERD), which can express the overall logical structure of a database graphically

Entities

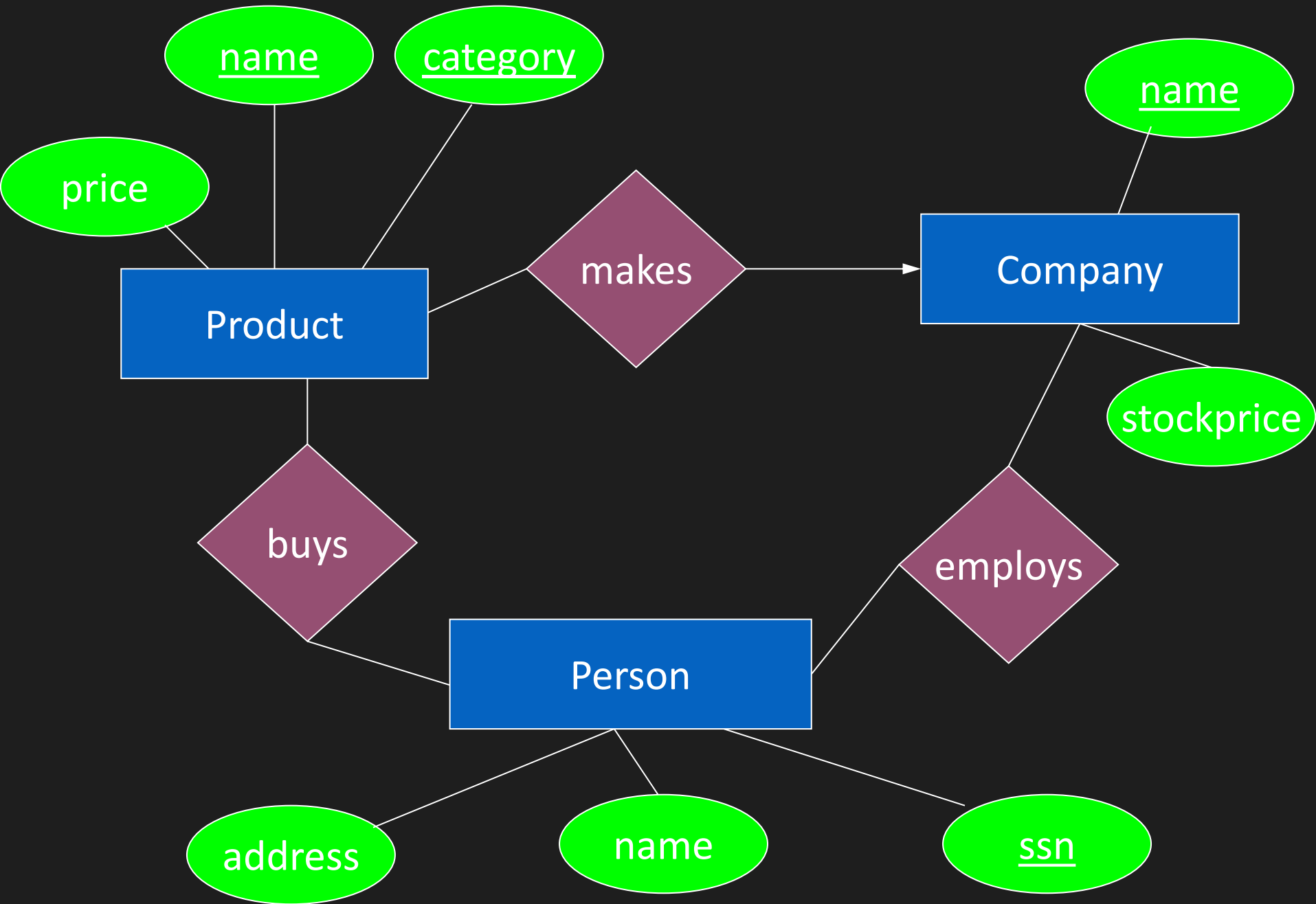
Actor

Attributes

name

Relationships between entities

buys





## CLASSES OF ATTRIBUTES

| הסבר                             |       | דוגמה           |               |
|----------------------------------|-------|-----------------|---------------|
| לא ניתנת לפיצול                  |       | תאריך לידה      | Simple        |
| ניתן לפצל אותה לעוד מאפיינים     |       | כתובת           | Composite     |
| ניתן לקבל את ערכה גם ממאפיין אחר | Age   | גיל             | Derived       |
| ערך אחד ויחיד לדוגמה מספר        |       | תז, מספר קטלוגי | Single-valued |
| יכול לקבל מספר ערכים             | Phone | מספר טלפון      | Multi-valued  |

# FIRST NORMAL FORM (1NF)

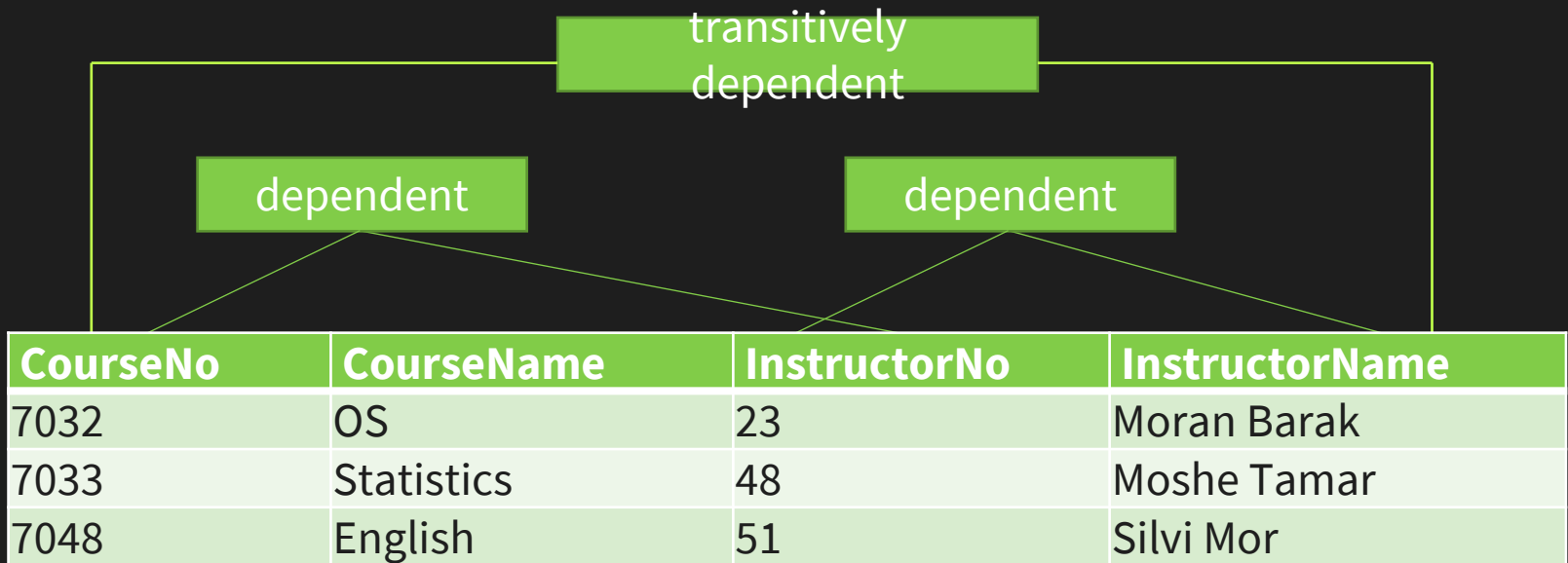
- זיהוי קבוצות שחוזרות על עצמן
- העברתן לטבלה חדשה
- הגדרת מפתח לטבלה החדשה

# SECOND NORMAL FORM (2NF)

- זיהוי עמודות שלא מבוססות על PK (במקרה של מפתח מורכב)
- העברתן לטבלה חדשה
- הגדרת מפתח לטבלה החדשה

# THIRD NORMAL FORM (3NF)

- זיהוי קשרים טרזטיבים ל PK
- העברתן לטבלה חדשה
- הגדרת מפתח לטבלה החדשה



# BOYCE CODD FORM (BCNF)

- זיהוי עמודות שיש בהן תלות שלא תלויה במפתח שגורמת להכפלת נתונים
- העברתן לטבלה חדשה
- הגדרת מפתח לטבלה החדשה